Fast Mining and Dynamic Time-to-Event Prediction over Multi-sensor Data Streams

Kota Nakamura
InfoTech, Toyota Motor Corporation
Tokyo, Japan
kota.nakamura.10@toyota.global

Koki Kawabata
SANKEN, Osaka University
Osaka, Japan
koki@sanken.osaka-u.ac.jp

Yasuko Matsubara
SANKEN, Osaka University
Osaka, Japan
yasuko@sanken.osaka-u.ac.jp

Yasushi Sakurai
SANKEN, Osaka University
Osaka, Japan
yasushi@sanken.osaka-u.ac.jp

Abstract

Given real-time sensor data streams obtained from machines, how can we continuously predict when a machine failure will occur? This work aims to continuously forecast the timing of future events by analyzing multi-sensor data streams. A key characteristic of real-world data streams is their dynamic nature, where the underlying patterns evolve over time. To address this, we present TimeCast, a dynamic prediction framework designed to adapt to these changes and provide accurate, real-time predictions of future event time. Our proposed method has the following properties: (a) *Dynamic:* it identifies the distinct time-evolving patterns (i.e., stages) and learns individual models for each, enabling us to make adaptive predictions based on pattern shifts. (b) *Practical:* it finds meaningful stages that capture time-varying interdependencies between multiple sensors and improve prediction performance; (c) *Scalable:* our algorithm scales linearly with the input size and enables online model updates on data streams. Extensive experiments on real datasets demonstrate that TimeCast provides higher prediction accuracy than state-of-the-art methods while finding dynamic changes in data streams with a great reduction in computational time.

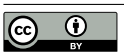
CCS Concepts

• **Information systems** → **Data stream mining**; **Information systems applications**.

Keywords

Streaming time-to-event prediction, Time series, Predictive maintenance, Patient monitoring

ACM Reference Format:
Kota Nakamura, Koki Kawabata, Yasuko Matsubara, and Yasushi Sakurai. 2026. Fast Mining and Dynamic Time-to-Event Prediction over Multi-sensor Data Streams . In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.1 (KDD '26), August 09–13, 2026, Jeju Island, Republic of Korea.* ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3770854.3780164



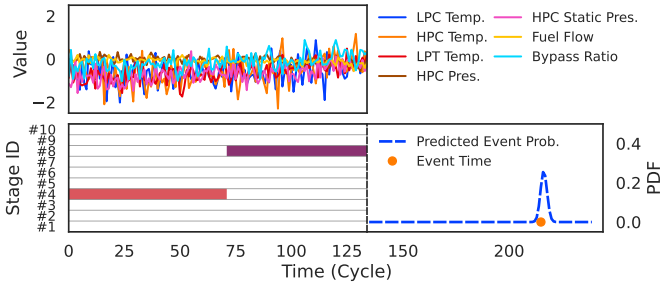
This work is licensed under a Creative Commons Attribution 4.0 International License.
KDD '26, Jeju Island, Republic of Korea
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2258-5/2026/08
https://doi.org/10.1145/3770854.3780164

1 Introduction

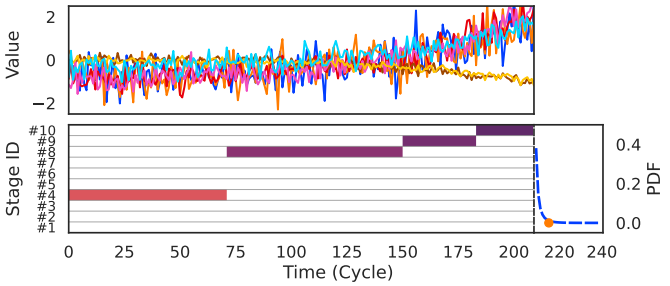
With the rapid growth in Internet of Things (IoT) deployment, real-time sensor data is being generated and collected by a wide range of applications [54], including automated factories [20], digital twins [44, 59], and electronic health record systems [28, 34], from which one of the most fundamental demands in data science and engineering is deriving actionable insights, such as predicting the timing of future machine failures or patient deaths. For example, a significant interest for industrial managers is obtaining more accurate estimates of failure time to schedule preventive maintenance that minimizes downtime and maximizes operational lifetime [29]. For patient monitoring in intensive care units (ICUs), it is essential to continuously estimate the time (i.e., risk) of a clinically critical event, such as death or the onset of disease, for better hospital resource management by focusing on patients that need it most [22]. To address these scenarios, we focus on an important yet challenging problem, namely, streaming time-to-event prediction, where our goal is to analyze real-time sensor sequences and continuously predict when a future event will occur.

Time-to-event prediction captures relationships between observations (e.g., sensor readings) and the time duration until an event of interest occurs. It can predict the event probabilities as a function of time, allowing us to flexibly assess the risk of event occurrence at any given time. In contrast, widely used binary classifiers predict whether the event of interest occurs after a predetermined duration (e.g., 30 seconds) and can only assess risk at a specific time [10, 35].

The problem of time-to-event prediction becomes more challenging when data arrives in a streaming or online manner. Assume that we have a sensor data stream $X_{:t_c}$, which is a time-evolving sequence of d-dimensional observations, i.e., $X_{:t_c} = \{x_1, \ldots, x_{t_c}\}$, where x_{t_c} is the most recent observation, and t_c increases with every new time tick. Such a situation requires an efficient algorithm that analyzes the continuously growing data stream and makes real-time predictions to design countermeasures as soon as the risk increases. Moreover, sensor data streams are usually non-stationary, changing their behavior over time. For example, in ICU patient monitoring, sensor measurements of vital signs change through distinct temporal phases, reflecting the patient's condition as it approaches clinically critical events [67]. These phases are key features that represent the temporal evolutions for entire data streams, which we specifically refer to as "*stages*" hereafter. Unlike existing time-to-event prediction approaches [3, 36, 38, 42, 62],



(a) Snapshots of streaming time-to-event prediction (t_c = 135)



(b) Snapshots of streaming time-to-event prediction (t_c = 210)

Figure 1: Prediction results of TimeCast over a machine failure-related sensor data stream. The method continuously detects/updates time-evolving stages. Then, it adaptively predicts event probabilities depending on the current stages.

which are static (as opposed to dynamic) and seek to predict the event time based on individual observations x_{t_c}, the ideal method should model time-evolving stages as the sequence-level features of $X_{:t_c}$. *So what are meaningful stages for time-to-event prediction?* We aim to find distinct temporal patterns that not only capture latent structural similarities in observations but also enhance prediction performance.

In this paper, we present TimeCast, a dynamic approach for time-to-event prediction over multi-sensor data streams. TimeCast is based on a sequential multi-model structure that identifies meaningful stages in data streams by jointly learning descriptive and predictive features. Thus, it can effectively predict event probabilities at future time points while adapting to stage shifts. In short, the problem we wish to solve is as follows.

Informal Problem 1. ***Given*** *a sensor data stream $X_{:t_c}$ for a machine/patient at risk of an event of interest occuring, which consists of observations until the current time t_c, i.e., $X_{:t_c} = \{x_1, \ldots, x_{t_c}\}$,*

- ***Find*** *time-evolving stages that improve prediction performance while identifying latent structural similarities in $X_{:t_c}$,*
- ***Predict*** *event probabilities at future time points,*

continuously and quickly in a streaming fashion.

Preview of Our Results. Figure 1 shows an example of TimeCast applied to turbofan jet engine data. The data consists of seven sensor readings, including temperatures and pressures, measured at every cycle. Given the real-time sensor data stream, TimeCast continuously provides predictions for the time to failure while capturing stages and their changes behind the data stream.

Figure 1 (a) shows the original data stream (top) and the result we obtained with TimeCast (bottom) at the current time t_c = 135. TimeCast firstly identifies stages and their changes in the sensor data stream observed up to the current time t_c. The figure illustrates that TimeCast detects a stage shift from Stage #4 to Stage #8 around the time t = 75 and recognizes Stage #8 as the current stage. Note that the original data does not exhibit any obvious patterns or stages. Meanwhile, our method finds the stages that contribute to prediction performance while revealing interdependencies between sensors, as elaborated in Section 3. Finally, TimeCast predicts the probabilities of a failure event by employing a stochastic time-to-event predictor associated with the current stage. The bottom part of Figure 1 (a) also shows the prediction results, where the dashed blue line represents the predicted event probabilities, and the orange dot indicates the actual time that a failure occurs. The result demonstrates that TimeCast accurately predicts the failure time, i.e., the predicted probabilities show a high value around the actual event time.

Figure 1 (b) shows the snapshots of TimeCast outputs at the current time t_c = 210, where TimeCast identifies the stage shifts (i.e., #4 → #8 → #9 → #10), and then adaptively makes predictions with the predictor for Stage #10. Here, the predicted event probabilities provide a relatively high value at the recent time, indicating a significant risk and suggesting the need for immediate shutdown or maintenance. Consequently, the method continuously detects/updates the shifting points in the sensor data streams and adaptively predicts the event time while switching predictors depending on the stages. As we will show in the experiments, our dynamic prediction approach improves prediction accuracy with a great reduction in prediction time.

Contribution. The main contributions of our paper are:

- *Dynamic prediction approach*: We propose a novel prediction approach, TimeCast, which captures stages behind non-stationary sequences and adaptively predicts the event probabilities at future time points.
- *Practicality*: By jointly learning descriptive and predictive features, TimeCast can make accurate predictions while revealing individual temporal patterns and time-varying interdependencies between sensors.
- *Scalability*: The computational time of our algorithm is linear in the data size, with fast convergence. It can process incoming data in an online manner.

Reproducibility. Our source code and datasets are available at [1].

Outline. The rest of this paper is organized in a conventional way. After introducing related studies in Section 2, we formally define our problems and present our model in Section 3. We then propose the algorithms in Section 4. We provide our experimental results in Section 5, followed by a conclusion in Section 6.

2 Related Work

The mining of time-stamped event data has attracted great interest in many fields [5, 13, 18, 23, 27, 30, 32, 41, 46]. Table 1 summarizes the relative advantages of our method in relation to five aspects, and only TimeCast meets all the requirements. Our work lies at the intersection of the following three categories.

Time-to-Event Prediction. Event prediction methods based on temporal point processes [7, 56], such as Hawkes process [16, 70] and cascade Poisson process [26, 58], can model dependencies between recurrent events, where they account for how the occurrence

Table 1: Capabilities of approaches.

	DeepSurv/++	HMM/++	TS2Vec	CubeScope	AC-TPC	TimeCast
Time-to-Event Prediction	✓		some		✓	✓
Time-Series Modeling		✓	✓	✓		✓
Non-Stationarity		✓		✓	✓	✓
Predictive Clustering					✓	✓
Streaming Time-to-Event Prediction						✓

of past events influences the probability of future events while capturing the nature of events over time. Differing from these methods, time-to-event prediction (also called survival analysis) [3, 42, 62] models the relationships between observations (e.g., sensor readings) and the remaining time until an event occurs. These methods map observations to parameters of a stochastic process for the event time, such as a Wiener process [39, 71]. Recent works have extended the classical Cox proportional hazards model [15] with neural networks [31, 38, 53, 65] to capture nonlinear relationships. Cox-Time [36] relaxes the proportionality assumption of the Cox model, improving flexibility for large-scale data sets. DeepHit [38] is capable of capturing multiple types of events and their completing risks. However, existing methods are static and are not intended to handle streams of time-varying observations. In contrast, our method is a dynamic prediction approach that can be aware of changes in data streams by incorporating time series modeling.

Time Series Modeling. Hidden Markov models (HMM) and other dynamic statistical models are extended to capture non-stationary sequences and dynamically changing trends, known as *concept drift* [43], by performing the simultaneous segmentation and clustering of the time series [24, 25, 45]. To identify the segments and clusters effectively, Gaussian graphical models (GGMs) and their variants [21, 51, 60, 61] capture the interdependencies of variables in each subsequence. StreamScope [33] extends a hierarchical HMM-based model to analyze data streams. DMM [50] is a GGM-based subsequence clustering method that can identify segments and clusters across multiple sequences. Although these approaches find segments and clusters by focusing only on the similarity of observations, our proposed method identifies them by simultaneously evaluating both similarity and prediction performance. Deep neural network (DNN) models, including representation learning methods [11, 40, 69] and transformer-based architectures [49, 68], provide end-to-end learning frameworks to capture the dynamics of sequences. However, these methods are not designed to update models according to time-evolving data streams.

Summarization and Clustering. Probabilistic generative models [4, 46, 47] have been used to analyze large-scale event data, from which they find meaningful clusters, such as progression stages [57, 66]. CubeScope [48] has the ability to summarize time-stamped event streams and capture distinct temporal clusters. However, these methods are incapable of modeling the relationships between clusters and future events or predicting the time to event. Predictive clustering [2, 6, 52] is a powerful technique for combining predictions on future outcomes with clustering. AC-TPC [37] is a deep learning approach for temporal predictive clustering. However, it is not designed to time-to-event prediction, as it does not incorporate the sequential connectivity of clusters.

Consequently, none of these studies focuses on fast and dynamic time-to-event prediction for non-stationary data streams.

Table 2: Symbols and definitions.

Symbol	Definition
V, W	Number of instances for learning and prediction process, respectively
X_v	Sensor sequence for v-th instance, i.e., $X_v = \{x_{v,1}, \ldots, x_{v,T_v}\}$
$x_{v,t}$	t-th multivariate observation in v-th instance, i.e., $x_{v,t} \in \mathbb{R}^d$
d	Number of sensor variables
$\tau_{v,t}$	Remaining time until an event of interest occurs for instance v at time t.
$\mathcal{D}$	Labeled collection, i.e., $\mathcal{D} = \{(X_{v,:t}, \tau_{v,t})\}_{v,t=1}^{V,T_v}$
$X_{w,:t}$	Sensor data stream for w-th instance i.e., $X_{w,:t} = \{x_{w,1}, \ldots, x_{w,:t}, \ldots\}$
K	Number of stage models
$\theta^{(k)}$	Stage model, i.e, $\theta^{(k)} = \{\mu^{(k)}, \Lambda^{(k)}, f^{(k)}, \sigma_B^{(k)}\}$
Θ	Stage model set, i.e, $\Theta = \{\theta^{(k)}\}_{k=1}^{K}$
$s_{v,t}$	Stage assignment for observations $x_{v,t}$, i.e., $s_{v,t} \in \{1, \ldots, K\}$
S	Stage assignment set
$\mathcal{F}$	Full parameter set of TimeCast, i.e., $\mathcal{F} = \{\Theta, S\}$

3 Proposed Model

In this section, we propose our model for streaming time-to-event prediction. We begin by introducing our formal problem definition, and then describe our model in detail.

3.1 Problem Formulation

Table 2 lists the main symbols that we use throughout this paper. Let us consider a collection of longitudinal sensor sequences, where multiple sensor readings are obtained from multiple instances (e.g., machines or patients), at every time point, that is, each entry is composed of the form *(instance, sensor, time)*. Our goal is to (a) learn a prediction model using V instances for whom the event of interest has occurred, and (b) continuously predict the future event time for W instances not observed during the learning process, where V and W indicate the number of instances.

(a) Model learning: We consider a set of sensor sequences $\{X_v\}_{v=1}^{V}$ for V instances that were measured until the event of interest occurred. Each sequence X_v comprises T_v sequential observations,

$$X_v = \begin{bmatrix} | & | & | & & | \\ x_{v,1} & x_{v,2} & x_{v,3} & \ldots & x_{v,T_v} \\ | & | & | & & | \end{bmatrix}, \tag{1}$$

where $x_{v,t} \in \mathbb{R}^d$ is the t-th multivariate observation in the v-th instance obtained from d-dimensional sensors [1] and T_v indicates the event time. We denote $X_{v,:t} = \{x_{v,1}, \ldots, x_{v,t}\}$ as the partial sequence observed up until the specific time t. Here, the label $\tau_{v,t}$ represents the time interval from the current time t to the event time T_v, i.e., $\tau_{v,t} = (T_v - t)$. In other words, the label $\tau_{v,t}$ indicates the remaining time until the event of interest occurs for the v-th instance at time t. Our aim is to learn a model $\mathcal{F}$ that can consistently predict the label $\tau_{v,t}$ for every time tick t and every instance v based on the sequential observations $X_{v,:t}$. More specifically, we want to predict the event probabilities as a function of time $p_{v,t}(\tau)$ to flexibly assess the risk of event occurrence at any given time. Therefore, letting $\mathcal{D} = \{(X_{v,:t}, \tau_{v,t})\}_{v,t=1}^{V,T_v}$ be a labeled collection, we formally define our first problem as follows:

[1] Without loss of generality, the observation $x_{v,t}$ can be set as sliding window features with a window size m, where we can employ $[x_{v,t-m}, \ldots, x_{v,t}]$ as the observation.

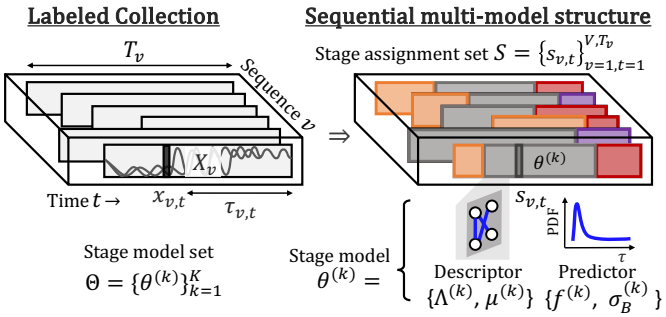


Figure 2: An overview of TimeCast for a labeled collection $\mathcal{D}$. The method is based on a sequential multi-model structure, which consists of a stage model set $\Theta = \{\theta^{(k)}\}_{k=1}^{K}$ and a stage assignment set S. It adopts a different stage model depending on time-varying behaviors. Each stage model $\theta^{(k)}$ consists of a descriptor $\{\Lambda^{(k)}, \mu^{(k)}\}$ and a predictor $\{f^{(k)}, \sigma_B^{(k)}\}$.

Problem 1. ***Given*** *a labeled collection, i.e., $\mathcal{D} = \{(X_{v,:}, \tau_{v,t})\}_{v,t=1}^{V,T_v}$,* ***Learn*** *a model $\mathcal{F}$ that maps each $X_{v,:}$ to the event probabilities as a function of time $p_{v,t}(\tau)$, i.e., $p_{v,t}(\tau) = \mathcal{F}(X_{v,:t})$.*

(b) Streaming time-to-event prediction: Once we have the model $\mathcal{F}$, we aim to achieve a time-to-event prediction over multiple data streams $\{X_w\}_{w=1}^{W}$, where each X_w is a continuously growing sequence. For the w-th instance, the stream observed up to the current time tick t_c is denoted as $X_{w,:t_c}$, where t_c increases with every new time tick. Formally, our second problem is as follows.

Problem 2. ***Given*** *a data stream $X_{w,:t_c}$ and the learned model $\mathcal{F}$,* ***Predict*** *the event probabilities as a function of time $p_{w,t_c}(\tau)$ at every new time tick t_c and* ***Update*** *the model $\mathcal{F}$ at every new instance X_w.*

3.2 TimeCast

We now present TimeCast model, $\mathcal{F}$, which is designed to satisfy the following properties for streaming time-to-event prediction:

- *Stochastic time-to-event predictor:* provides event probabilities for each observation by capturing the underlying stochastic process.
- *Interdependency-based descriptor*: characterizes each observation based on statistical interdependencies between sensors.
- *Sequential multi-model structure:* captures dynamic changes in sequences and enables adaptive prediction through multiple sequentially connected models.

Figure 2 shows an overview of TimeCast model for a labeled collection $\mathcal{D}$. The predictor and the descriptor are the building blocks of each model, which we refer to as a stage model. Each stage model is associated with a specific stage in $\mathcal{D}$. The sequential multi-model structure consists of multiple stage models that are sequentially connected. Details are provided in the remaining subsections.

3.2.1 Stochastic Time-to-Event Predictor. We begin with the simplest case, in which we model only a single stage. The first problem is to predict the event probabilities as a function of time $p_{v,t}(\tau)$ for a given observation $x_{v,t}$. We employ the concept of first hitting time, where the event time is defined as the first time the underlying progression process reaches a prescribed boundary. Specifically, we define the progression process $W(\tau)$ as a Wiener process, allowing us to represent the first hitting time as an inverse Gaussian distribution [12, 14]. The progression process $W(\tau)$ is written as the following stochastic differential equation:

$$dW(\tau) = \nu d\tau + \sigma_B dB(\tau), \tag{2}$$

where ν is the drift parameter capturing the rate of progression, σ_B is the diffusion parameter representing the uncertainty of the progression, and $\{B(\tau)|\tau \geq 0\}$ is a standard Brownian motion. That is, for each $\tau \geq 0$, $\sigma_B dB(\tau) \sim \mathcal{N}(0, \sigma_B^2 d\tau)$, which indicates that uncertainty increases as τ increases.

Since a true progression process is unobservable, we estimate the progression process $W(\tau)$ from observation $x_{v,t}$. Specifically, we introduce a link function[2] f which maps each $x_{v,t}$ to the event time $\tau_{v,t}$ and the progression process $W(\tau)$ is written as follows:

$$W(\tau) = \nu\tau + \sigma_B B(\tau), \ \nu = \frac{1}{f(x_{v,t})}. \tag{3}$$

Finally, we want to estimate the probabilities as a function of time $p_{v,t}(\tau)$ that the progression process $W(\tau)$ reaches a boundary c. Since Equation (3) ensures that the drift parameter satisfies $0 < \nu < 1$ and that the boundary $c = 1$, the event probabilities $p_{v,t}(\tau)$ is written as follows:

$$p_{v,t}(\tau; f, \sigma_B, x_{v,t}) = \frac{1}{\sqrt{2\pi\sigma_B^2\tau^3}} \exp\Big[-\frac{\big(1 - \frac{\tau}{f(x_{v,t})}\big)^2}{2\sigma_B^2\tau}\Big]. \tag{4}$$

Here, we refer to f and σ_B as a *predictor*, which enables us to predict the event probabilities $p_{v,t}(\tau)$ for a given observation $x_{v,t}$.

3.2.2 Interdependency-based descriptor. The next question is how to characterize each observation $x_{v,t}$ to identify changes over sensor sequences. Real sensor sequences might contain various types of noise that can distort observed values. To maintain robustness against noise, we focus on interdependencies between sensors rather than individual statistics. Specifically, we employ Gaussian graphical models (GGMs), which model the conditional independence between d sensor variables in an observation $x_{v,t} \in \mathbb{R}^d$. The model captures the underlying distribution of each observation $x_{v,t}$, i.e., $x_{v,t} \sim \mathcal{N}(\mu, \Lambda^{-1})$, where $\mu \in \mathbb{R}^d$ and $\Lambda \in \mathbb{R}^{d\times d}$ indicate the mean and the *sparse* precision matrix, respectively. Each value of the sparse precision matrix $\Lambda_{i,j}$ can indicate pairwise conditional independence, that is,

$$\Lambda_{i,j} = 0 \Leftrightarrow x_{v,t,i} \perp\!\!\!\perp x_{v,t,j} \mid x_{v,t,\setminus\{i,j\}}, \tag{5}$$

where $x_{v,t,i}$ denotes the i-th sensor variable in observation $x_{v,t}$, resulting in the sparse precision matrix Λ being interpreted as the adjacency matrix of a graph that describes the interdependencies. Here, we refer to μ and Λ as a *descriptor*, which characterizes a given observation $x_{v,t}$.

3.2.3 Sequential Multi-Model Structure. Thus far, we have discussed *predictor* and *descriptor*, which provides event probabilities as a function of time and interdependency-based representations for each observation $x_{v,t}$. However, the model focuses only on individual observations and remains insufficient for capturing whole sensor sequences, containing various types of stages. We thus propose a sequence-level model architecture.

[2]In our experiments, we used an orthogonal projection for f, which has a closed-form solution with the weights A, i.e., $\tau = Ax$. However, the link function is not constrained, and exploring the potential improvements remains an open problem for future work.

Definition 1 (Stage model set: Θ). *Let Θ be a set of K stage models. The k-th stage model $\theta^{(k)}$ consists of a predictor and a descriptor, i.e., $\theta^{(k)} = \{f^{(k)}, \sigma_B^{(k)}, \mu^{(k)}, \Lambda^{(k)}\}$.*

Our model employs a different stage model $\theta^{(k)} \in \Theta$ that depends on a time-varying stage. Thus, we also want to determine the assignments of stage models for each observation $x_{t,v}$.

Definition 2 (Stage assignment set: S). *Let S be a set of stage assignments $s_{v,t}$. The stage assignment $s_{v,t}$ represents a stage index for each observation $x_{v,t}$, i.e., $s_{v,t} \in \{1, \dots, K\}$, and is constrained by the sequential connectivity as follows:*

$$t < t' \Rightarrow s_{v,t} \leq s_{v,t'} \mid \forall v, t, t'. \tag{6}$$

Note that the sequential connectivity of Eq (6) enforces that the stage assignments $s_{v,t}$ never decrease over time, providing two key benefits. *First,* it ensures that stage assignments are interpreted as an irreversible progression, analogous to real-world processes leading to future events such as disease progression [67] and machine degradation [29]. *Second,* it maintains temporal consistency by ignoring abrupt fluctuations and repeated changes in stage assignments.

Consequently, the complete parameter set of TimeCast that we want to estimate is as follows.

Definition 3 (Full Parameter Set of TimeCast: $\mathcal{F}$). *Let $\mathcal{F}$ be a complete set of TimeCast, i.e., $\mathcal{F} = \{\Theta, S\}$, where Θ indicates a stage model set and S indicates a stage assignment set.*

4 Optimization Algorithms

Thus far, we have introduced our mathematical concept of TimeCast. Next, we tackle Problem 1 and Problem 2 by proposing the following two algorithms.

- Learning algorithm for Problem 1: Efficiently find the optimal parameter of $\mathcal{F}$ for a given labeled collection $\mathcal{D}$.
- Streaming algorithm for Problem 2: Adaptively predict the event probabilities $p_{w,t_c}(\tau)$ for a data stream $X_{w,:t_c}$.

We first introduce our objective function and then describe the proposed algorithms in detail.

Objective Function. Given a labeled collection $\mathcal{D}$, we aim to estimate the full parameter set $\mathcal{F} = \{\Theta, S\}$ that maximizes the following objective:

$$\underset{\Theta, S \nearrow t}{\operatorname{argmax}} \sum_{k=1}^{K} \underbrace{\Psi_d(\mathcal{D} \mid \theta^{(k)}, S)}_{\text{Descriptor}} + \beta \underbrace{\Psi_p(\mathcal{D} \mid \theta^{(k)}, S)}_{\text{Predictor}}, \tag{7}$$

where $S \nearrow t$ is a constraint of sequential connectivity in Eq. (6) and $\beta \geq 0$ is a coefficient chosen to balance the descriptors and the predictors. The first term is the objective function of the descriptor for each stage k:

$$\Psi_d(\mathcal{D} \mid \theta^{(k)}, S) = \sum_{s_{v,t}=k} \left[\psi_d(x_{v,t} \mid \mu^{(k)}, \Lambda^{(k)}) \right] - \alpha ||\Lambda^{(k)}||_{od,1},$$

$$\psi_d(x_{v,t} \mid \mu^{(k)}, \Lambda^{(k)}) = -\frac{1}{2}(x_{v,t} - \mu^{(k)})^{\mathrm{T}} \Lambda^{(k)} (x_{v,t} - \mu^{(k)}) + \frac{1}{2} \log\det \Lambda^{(k)} - \frac{d}{2} \log(2\pi), \tag{8}$$

where ψ_d is the Gaussian log likelihood that $x_{v,t}$ comes from stage k, and $||\cdot||_{od,1}$ is the off-diagonal ℓ_1-norm, which enforces element-wise sparsity for the precision matrix, regulated by the trade-off parameter $\alpha \geq 0$. The second term is the log likelihood of the predictor for each stage k (up to a constant and scale):

$$\Psi_p(\mathcal{D} | \theta^{(k)}, S) = \sum_{s_{v,t}=k} \psi_p(x_{v,t}, \tau_{v,t} \mid f^{(k)}, \sigma_B^{(k)}),$$

$$\psi_p(x_{v,t}, \tau_{v,t} \mid f^{(k)}, \sigma_B^{(k)}) = -||\tau_{v,t} - f^{(k)}(x_{v,t})||_2 - \log(\sigma_B^2) - \frac{1}{\sigma_B^2} || \frac{1}{\tau_{v,t}} - \mu_\tau^{(k)} ||_2, \tag{9}$$

where $||\cdot||_2$ denotes ℓ_2-norm and $\mu_\tau^{(k)}$ denotes the mean of the increments in the k-th progression process.

Notably, the objective function simultaneously evaluates both descriptive quality and prediction accuracy, so the resulting parameters $\{\Theta, S\}$ reflect both aspects of the data. This design follows the concept of multi-task learning [9], where the joint learning of multiple tasks serves as an inductive bias that improves generalization. As discussed in Section 5, the joint learning with both parts improved prediction performance in our experiments.

Algorithm 1 Learning Algorithm $(\mathcal{D}, K, \alpha, \beta)$

Input: (a) Labeled collection $\mathcal{D} = \{(X_{v,:t}, \tau_{v,t})\}_{v,t=1}^{V,T_v}$
(b) Initial number of stages K
(c) Sparse parameter α
(d) Balance parameter β
Output: Full parameter set $\mathcal{F}$
1: $\{\Theta, S\} \leftarrow$ Initialize$(\mathcal{D}, K, \alpha, \beta)$;
2: **repeat**
3: **for** k-th stage **do**
4: $\Theta \leftarrow$ UpdateStageModels$(\mathcal{D}, S, \alpha)$; // Section 4.1.1
5: **end for**
6: **for** v-th sequence **do**
7: $S \leftarrow$ UpdateAssignments(X_v, Θ, β); // Section 4.1.2
8: **end for**
9: **until** convergence;
10: **return** $\mathcal{F}$;

4.1 Learning Algorithm

Our first goal is to estimate the full parameter set $\mathcal{F}$ to maximize the objective function in Eq. (7). However, this problem is combinatorial and non-differentiable, rendering widely-used SGD-based methods inapplicable. Instead, we propose an efficient learning algorithm that exhibits stable and monotonic optimization behavior. Algorithm 1 shows the overall procedure, where we first initialize $\{\Theta, S\}$ to some random values and then iteratively update subsets of parameters. First, we update the stage model set Θ while keeping the stage assignments S fixed (lines 3-5). Second, we update S with the fixed parameters of Θ (lines 6-8). We iterate these two steps until convergence. We describe each step in detail in the following subsections.

4.1.1 UpdateStageModels. In this step, we estimate the parameters for all stage models $\{\theta^{(k)}\}_{k=1}^K$, while fixing the stage assignments S. Once the stage assignments are fixed, we can optimize each stage $\theta^{(k)}$ independently by maximizing Eq. (8) and Eq. (9). Specifically, maximizing Eq. (8) is equivalent to the graphical lasso problem [17]. Since this is a convex optimization problem, we use the alternating direction method of multipliers [8], which efficiently converges on the globally optimal solution. In addition, we maximize Eq. (9) through maximum likelihood estimation. The details of each maximization problem are described in Appendix A.

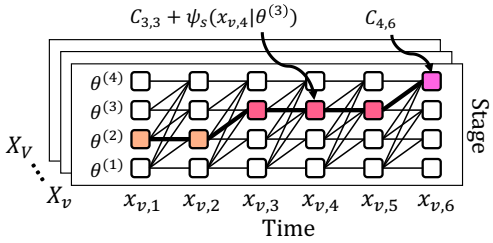


Figure 3: Dynamic programming algorithm for stage assignments. The algorithm efficiently finds the optimal stage assignments by sequentially computing the cost $C_{k,t}$.

4.1.2 UpdateAssignments. In this step, we find the optimal stage assignments S, while fixing the value of Θ. We rewrite our maximization problem (i.e., Eq. (7)) in terms of the stage assignments S, which is written as follows for each sequence X_v:

$$\underset{s_{v,t} \nearrow t}{\operatorname{argmax}} \sum_{t=1}^{T_v} \psi_s(x_{v,t} \mid \theta^{(s_{v,t})}), \tag{10}$$

$$\psi_s(x_{v,t} \mid \theta^{(s_{v,t})}) = \psi_d(x_{v,t} \mid \mu^{(s_{v,t})}, \Lambda^{(s_{v,t})}) + \beta\psi_p(x_{v,t}, \tau_{v,t} \mid f^{(s_{v,t})}, \sigma_B^{(s_{v,t})}).$$

Since each stage assignment $s_{v,t}$ is constrained by the sequential connectivity in Eq. (6), Eq. (10) is a combinatorial optimization problem that requires finding the optimal assignments of K stage models to T_v observations. However, the number of possible assignments is $O(K^{T_v})$, making it computationally prohibitive. Therefore, we introduce a dynamic programming algorithm that finds a globally optimal solution in only $O(K^2 T_v)$ operations. Specifically, we sequentially compute a cost $C_{k,t}$, which is provided as follows:

$$C_{k,t} = \begin{cases} \psi_s(x_{v,1} \mid \theta^{(k)}) & (t = 1) \\ \max_{1 \le k' \le k}\{C_{k',t-1}\} + \psi_s(x_{v,t} \mid \theta^{(k)}) & (2 \le t \le T_v) \end{cases} \tag{11}$$

When computing the cost $C_{k,t}$, we also record the path of the stage assignments. After computing C_{k,T_v} for all stages K, we can find the optimal stage assignments by choosing the path that gives the maximum cost, i.e., $\max_{1 \le k \le K}\{C_{k,T_v}\}$. This procedure is illustrated as a lattice diagram in Figure 3, where the stages are on the vertical axis and the time on the horizontal axis. At each time tick t, $C_{k,t}$ provides the path that maximizes the cost to reach the k-th stage among possible assignments.

Overall, our learning algorithm iteratively updates the stage models and assignments. Each update improves the objective in a monotonic manner, leading to a stable optimization process.

Lemma 1 (Proof in Appendix A.1). *The time complexity of the learning algorithm in TimeCast is $O(\#iter \cdot \sum_v T_v)$.*

4.2 Streaming Algorithm

We now address Problem 2, namely streaming time-to-event prediction. Assuming an observation x_{w,t_c} is continuously obtained as the current observation of a data stream for the w-th instance $X_{w,:t_c}$, our aim is to predict event probabilities $p_{w,t_c}(\tau)$ while updating the stage model set Θ to maintain prediction performance. Algorithm 2 outlines the overall procedure, which consists of two steps:

(1) AdaptivePredict: Continuously estimates the current stage s_{w,t_c} from the current observation x_{w,t_c} in the context of the data stream $X_{w,:t_c}$. The current stage s_{w,t_c} is identified by maximizing

Algorithm 2 Streaming Algorithm $(x_{w,t_c}, \Theta, \{C_{k,t_c-1}\}_{k=1}^K)$

Input: (a) Recent observation x_{w,t_c}
(b) Stage model set Θ
(c) Previous cost set $\{C_{k,t_c-1}\}_{k=1}^K$
Output: (a) Predicted event probabilities $p_{w,t_c}(\tau)$
(b) Updated stage model set Θ
(c) Updated cost set $\{C_{k,t_c}\}_{k=1}^K$

```
p_{w,t_c}(τ), {C_{k,t_c}}_{k=1}^K ← AdaptivePredict(x_{w,t_c}, Θ, {C_{k,t_c-1}}_{k=1}^K);
if t_c == T_w then // OnlineModelUpdate
    θ⁺ ← Initialize(X_w); Θ⁺ ← {Θ, θ⁺};
    repeat
        S ← UpdateAssignments(X_w, Θ⁺, β);
        Θ⁺ ← UpdateStageModels-Online(X_w, S, α, Θ⁺);
    until convergence;
    if Θ⁺ improves prediction accuracy then
        Θ ← Θ⁺
    end if
end if
return p_{w,t_c}(τ), Θ, {C_{k,t_c}}_{k=1}^K;
```

the following equation:

$$s_{w,t_c} \leftarrow \underset{\{s_{w,t}\} \nearrow t}{\operatorname{argmax}} \sum_{t=1}^{t_c} \psi_d(x_{w,t} \mid \mu^{(s_{w,t})}, \Lambda^{(s_{w,t})}). \tag{12}$$

Although estimating the current stage s_{w,t_c} requires past observations $\{x_{w,1}, \ldots, x_{w,t_c-1}\}$, accessing them at each prediction step is computationally expensive in streaming environments. Therefore, we solve Eq. (12) in an online manner. Similar to Eq. (11), we use a dynamic programming algorithm, which is written as follows:

$$C_{k,t_c} = \begin{cases} \psi_d(x_{w,t_c} \mid \mu^{(k)}, \Lambda^{(k)}), & (t_c = 1) \\ \max_{1 \le k' \le k}\{C_{k',t_c-1}\} + \psi_d(x_{w,t_c} \mid \mu^{(k)}, \Lambda^{(k)}) & (2 \le t_c) \end{cases}$$

$$s_{w,t_c} \leftarrow \underset{1 \le k \le K}{\operatorname{argmax}} C_{k,t_c}. \tag{13}$$

This procedure enables us to estimate the current stage s_{w,t_c} based on the current observation x_{w,t_c} and the cost set $\{C_{k,t_c-1}\}_{k=1}^K$ at previous time $t_c - 1$. For the next time tick $t_c + 1$, we retain the cost set $\{C_{k,t_c}\}_{k=1}^K$ and discard the cost set $\{C_{k,t_c-1}\}_{k=1}^K$. Finally, the algorithm predicts the event probabilities as a function of time $p_{w,t_c}(\tau)$ by exploiting the stage model $\theta^{(s_{w,t_c})}$.

(2) OnlineModelUpdate: Runs when the data stream $X_{w,:t_c}$ is observed up to the event time T_w, i.e., when $\{(X_{w,:t}, \tau_{w,t})\}_{t=1}^{T_w}$ are available. To adapt non-stationary data streams, this step employs a generate-and-validate approach that generates a new stage model set Θ^+ and adopts the models only if this leads to improved prediction accuracy. Specifically, we first initialize the new stage model θ^+ based on observations assigned to the stage with the worst prediction accuracy. An augmented model set $\Theta^+ = \{\Theta, \theta^+\}$ is updated by iterating UpdateAssignments and UpdateStageModels. Given the existing model set Θ and the augmented model set Θ^+, we compare their prediction accuracies on X_w. Note that estimating the existing stage models $\{\theta^{(k)}\}_{k=1}^K$ requires observations assigned to each stage in the learning algorithm. Owing to the careful design of the stage models based on the means and covariances of observations, the parameters of $\{\theta^{(k)}\}_{k=1}^K$ can be updated online using Welford's algorithm [64].

Lemma 2 (Proof in Appendix A.2). *The time complexity of the streaming algorithm in TimeCast is $O((1 + \#iter) \cdot K^2)$ amortized per time step.*

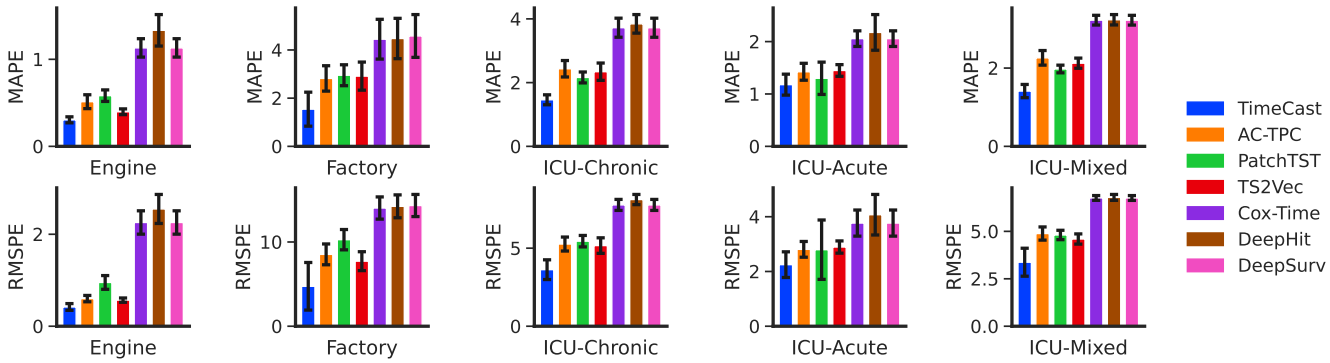


Figure 4: Comparison of prediction performance. TimeCast consistently outperforms its baselines (lower is better).

Table 3: Dataset description

Dataset	$V+W$	d	$\sum_v T_v$	$Avg(T_v)$
Industrial dataset: *(machine, sensor, time)* → Failure				
#1 *Engine*	200	7	45,351	227 ± 72
#2 *Factory*	98	4	89,538	914 ± 710
Medical dataset: *(patient, vital sign, time)* → Mortality				
#3 *ICU-Chronic*	355	6	98,320	277 ± 217
#4 *ICU-Acute*	112	6	20,193	180 ± 96
#5 *ICU-Mixed*	521	6	141,336	271 ± 194

5 Experiments

In this section, we evaluate the performance of TimeCast. We answer the following questions through experiments.

- (Q1) *Accuracy*: How accurately does it achieve streaming time-to-event prediction?
- (Q2) *Scalability*: How does it converge and scale in terms of computational time?
- (Q3) *Real-world Effectiveness*: How does it provide meaningful discoveries through stage identification?

Experimental Settings. We use five publicly available real-world datasets listed in Table 3, consisting of sensor sequences recorded until a particular event occurs in mechanical systems and patients at ICUs. The six baseline methods are as follows: DeepSurv [31], DeepHit [38], and Cox-Time [36], which are time-to-event prediction methods. We also compared our method with TS2Vec [69], a time series representation learning method; PatchTST [49], a transformer-based time-series modeling approach; and AC-TPC [37], a predictive clustering method. We use scale-invariant performance metrics, MAPE and RMSPE, based on percentage errors between the predicted event time $\hat{\tau}_{w,t}$ and the true event time $\tau_{w,t}$. Although the true event time $\tau_{w,t}$ can be a larger value depending on the sequence length, these metrics allow for consistent comparison across different scales. Lower values indicate better prediction accuracy. For evaluations, we apply 5-fold cross validation. We randomly separated the instances into a training set (80%) and a testing set (20%). We reserved 10% of the training set as a validation set. The hyperparameters were selected based on the prediction performance on the validation set. We used the parameters of TimeCast for $\alpha = 1$, $\beta = 0.1$, and $K = 5$. Detailed experimental settings, including data preprocessing and baseline parameters, are provided in Appendix B.

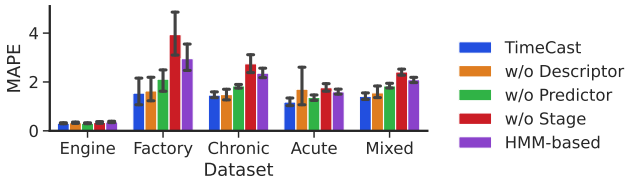


Figure 5: Prediction accuracy of TimeCast and its variants on MAPE. Each component improves the prediction performance on all datasets (lower is better).

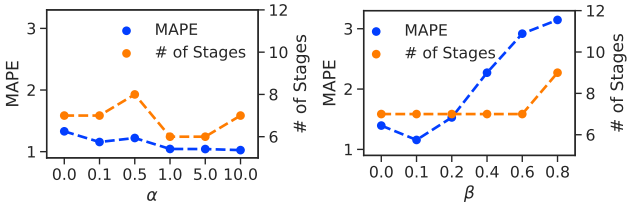


Figure 6: Hyperparameter sensitivity of TimeCast.

Q1. Accuracy. We compared the prediction performance of TimeCast with that of the baselines. Figure 4 shows the MAPE and RMSPE on all the datasets. For the methods that provide the event probabilities $p_{w,t}(\tau)$, we employ the mean of $p_{w,t}(\tau)$ as the predicted event time $\hat{\tau}_{w,t}$. Our method consistently outperforms its baselines because it can capture non-stationary data streams through a sequential multi-model structure. DeepSurv, DeepHit, and Cox-Time are static time-to-event prediction methods that focus only on individual observations. They fail to capture dynamic changes and sequential features for given data streams. Although TS2Vec and PatchTST effectively learn the contextual representation of sequences, they cannot distinguish multiple stages. AC-TPC is a predictive clustering method that makes predictions while finding clusters. However, the method is capable of modeling sequential connectivity between clusters, leading to suboptimal results in streaming time-to-event prediction.

Ablation Study. To verify the effectiveness of the proposed components in TimeCast, we conducted ablation studies on all the datasets. Figure 5 shows the prediction accuracy of TimeCast and its variants, which learn $\mathcal{F}$ while excluding the effect of a specific component. Specifically, **(a) w/o Stage** removes the sequential connectivity of Eq. (6), **(b) w/o Predictor** removes the effect of the predictor (i.e., $\beta = 0$), **(c) w/o Descriptor** learns the stage models without imposing sparsity on the precision matrices (i.e., $\alpha = 0$), and

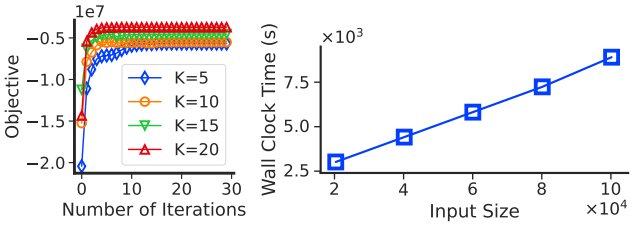


Figure 7: Scalability of TimeCast. (left) Fast convergence of our learning algorithm. It converged within 20 iterations in the *ICU-Acute* dataset. (right) Wall clock time vs. input size. The learning algorithm of TimeCast scales linearly.

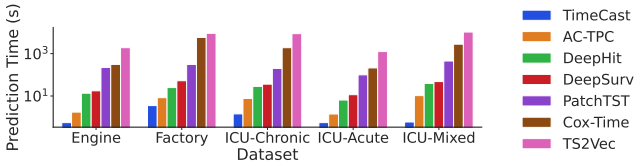


Figure 8: Prediction time for all datasets. TimeCast consistently faster than its baselines. The results are shown in log scale.

(d) HMM-based uses HMM to find stages instead of the proposed stage models. The results show that the proposed components are complementary, i.e., joint optimization with all parts improves the prediction performance.

Hyperparameter Sensitivity. We analyze the sensitivity of TimeCast to its hyperparameters. Figure 6 shows the prediction results when varying hyperparameter settings on the *ICU-Acute* dataset. In the left part of Figure 6, a larger value of the sparsity parameter α leads to learn more robust descriptors against noise, resulting in a slight improvement in MAPE. A detailed analysis of the effect of sparsity is provided in Appendix B.3. The parameter β indicates the effects of the predictors in both the learning algorithms and online model updates. The right part of Figure 6 shows that the larger value of β degrades the MAPE while only marginally affecting the number of stages.

Q2. Scalability. We evaluate the efficiency of TimeCast. We first show that the learning algorithm converges within a small number of iterations. The left part of Figure 7 shows the value of our objective function (i.e., Eq. (7)) in each iteration on the *ICU-Acute* dataset. Thanks to our efficient optimization, even with 20 stages, it converged within 20 iterations. The right part of Figure 7 shows the computational time for the learning algorithm when we vary the total duration of the sequences on the *ICU-Mixed* dataset. Since it takes $O(\#iter \cdot \sum_v T_v)$ time (as discussed in Lemma 1) and $\#iter$ is small in practice, the complexity scales linearly with respect to the data length. Figure 8 compares the prediction time with its competitors as regards computational time on all the datasets. Our method outperforms its baselines in speed by up to four orders of magnitude, enabling rapid response even when sensor readings arrive at high rates.

Q3. Real-World Effectiveness. The prediction results for the *Engine* dataset have already been presented in Figure 1. The figure demonstrated that TimeCast continuously provides the probabilities for future failure time, leading to immediate shutdown and maintenance scheduling. We here provide some of our discoveries

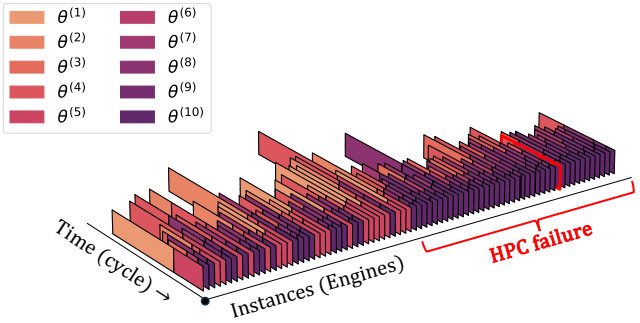


Figure 9: Stage identification of TimeCast. The method identifies ten stages, shown as colored segments. Here, the failure-specific evolution pattern appears on multiple instances, highlighted by the red bracket.

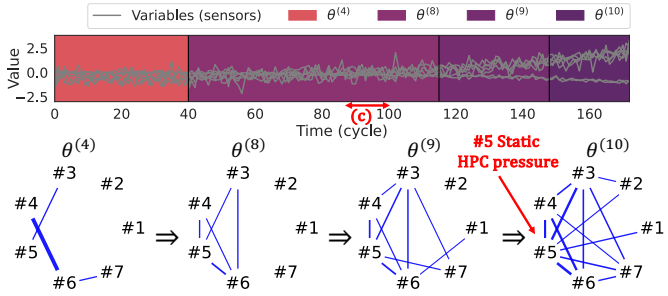


Figure 10: Time-varying interdependencies between seven sensor variables. TimeCast finds that variations in the other sensors depend on the static pressure at the HPC just before HPC failure.

on stage identification of TimeCast, which allow us to understand how the conditions of turbofan engines change over time.

Figure 9 shows stage identification for multiple instances (i.e., engines). TimeCast discovers ten stages (i.e., $\theta^{(1)}, \ldots, \theta^{(10)}$) and their shifting points, where the assignments of each stage are indicated as a set of colored segments, and each engine is aligned along the failure time. Here, we observed failure-specific behavior: multiple engines indicated by a red bracket have similar evolutionary stages (i.e., $\theta^{(4)} \to \theta^{(8)} \to \theta^{(9)} \to \theta^{(10)}$). According to the investigation of the dataset [55], these engines experienced the same type of failure called high-pressure compressor (HPC) failure.

Each stage model $\theta^{(k)}$ captures interdependencies between sensor variables. Figure 10 illustrates the time-varying interdependencies for an engine that experienced HPC failure, where the dependencies are visualized as a graph. The nodes indicate individual sensor variables, and the edge widths indicate the connection intensities. Notably, the number of edges of #5 (static pressure at the HPC) gradually increases and is connected to every other node in the last stage. This means that just before HPC failure, variations in the other sensors depend on the static pressure at the HPC.

6 Conclusion

In this paper, we focused on streaming time-to-event prediction and presented TimeCast, which exhibits all the desirable properties that we listed in the introduction; **(a) Dynamic**: TimeCast

adaptively provides event probabilities at future time points while detecting and updating stage shifts in data streams. **(b) Practical**: The method continuously predicts the event time with high accuracy while providing semantic information about the data. **(c) Scalable**: The learning algorithm showed fast convergence and linear scalability with respect to data size. The streaming algorithm makes predictions while efficiently updating the model structure. Our experimental evaluation using five real datasets showed that TimeCast outperforms existing methods in terms of accuracy and execution speed. The framework of TimeCast is general and flexible, opening up new possibilities for time-to-event prediction over data streams. Exploring alternative models for predictors and descriptors, as well as extending the framework to other application domains, constitutes an important direction for future research.

Acknowledgments

The authors would like to thank the anonymous referees for their valuable comments and helpful suggestions. This work was partly supported by JSPS KAKENHI Grant-in-Aid for Scientific Research Number JP20910053, JP25K21208 JST CREST JPMJCR23M3, JST START JPMJST2553, JST CREST JPMJCR20C6, JST K Program JPMJKP25Y6, JST COI-NEXT JPMJPF2009, JST COI-NEXT JPMJPF2115, the Future Social Value Co-Creation Project - Osaka University.

References

[1] 2026. *TimeCast*. https://github.com/kotaNakm/TimeCast.
[2] Henrique Aguiar, Mauro Santos, Peter Watkinson, and Tingting Zhu. 2022. Learning of cluster-based feature importance for electronic health record time-series. In *International Conference on Machine Learning*. PMLR, 161–179.
[3] Sattar Ameri, Mahtab J Fard, Ratna B Chinnam, and Chandan K Reddy. 2016. Survival analysis based framework for early prediction of student dropouts. In *Proceedings of the 25th ACM international on conference on information and knowledge management*. 903–912.
[4] Alex Beutel, Kenton Murray, Christos Faloutsos, and Alexander J Smola. 2014. Cobafi: collaborative bayesian filtering. In *WWW*. 97–108.
[5] Siddharth Bhatia, Mohit Wadhwa, Kenji Kawaguchi, Neil Shah, Philip S Yu, and Bryan Hooi. 2023. Sketch-based anomaly detection in streaming graphs. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 93–104.
[6] Hendrik Blockeel, S Dzeroski, Jan Struyf, and Bernard Zenko. 2019. Predictive Clustering.
[7] Tanguy Bosser and Souhaib Ben Taieb. 2023. On the predictive accuracy of neural temporal point process models for continuous-time event data. *arXiv preprint arXiv:2306.17066* (2023).
[8] Stephen Boyd, Neal Parikh, Eric Chu, Borja Peleato, Jonathan Eckstein, et al. 2011. Distributed optimization and statistical learning via the alternating direction method of multipliers. *Foundations and Trends® in Machine learning* 3, 1 (2011), 1–122.
[9] Rich Caruana. 1997. Multitask learning. *Machine learning* 28 (1997), 41–75.
[10] Zheng Chen, Yasuko Matsubara, Yasushi Sakurai, and Jimeng Sun. 2025. Long-term eeg partitioning for seizure onset detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 14221–14229.
[11] Ziqiang Cheng, Yang Yang, Wei Wang, Wenjie Hu, Yueting Zhuang, and Guojie Song. 2020. Time2graph: Revisiting time series modeling with dynamic shapelets. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 3617–3624.
[12] Raj S. Chhikara and J. Leroy Folks. 1989. *The Inverse Gaussian Distribution: Theory, Methodology, and Applications*. Marcel Dekker, Inc., USA.
[13] Naoki Chihara, Yasuko Matsubara, Ren Fujiwara, and Yasushi Sakurai. 2025. Modeling Time-evolving Causality over Data Streams. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 153–164.
[14] D.R. Cox and H.D. Miller. 1965. *The Theory of Stochastic Processes*. Methuen.
[15] David R Cox. 1972. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)* 34, 2 (1972), 187–202.
[16] Manisha Dubey, PK Srijith, and Maunendra Sankar Desarkar. 2023. Time-to-Event Modeling with Hypernetwork based Hawkes Process. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3956–3965.
[17] Jerome Friedman, Trevor Hastie, and Robert Tibshirani. 2007. Sparse inverse covariance estimation with the graphical lasso. *Biostatistics* 9, 3 (12 2007), 432–441. https://doi.org/10.1093/biostatistics/kxm045 arXiv:https://academic.oup.com/biostatistics/article-pdf/9/3/432/17742149/kxm045.pdf
[18] Ren Fujiwara, Yasuko Matsubara, and Yasushi Sakurai. 2025. Modeling Latent Non-Linear Dynamical System over Time Series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 11663–11671.
[19] Ary L Goldberger, Luis AN Amaral, Leon Glass, Jeffrey M Hausdorff, Plamen Ch Ivanov, Roger G Mark, Joseph E Mietus, George B Moody, Chung-Kang Peng, and H Eugene Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: components of a new research resource for complex physiologic signals. *circulation* 101, 23 (2000), e215–e220.
[20] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. 2013. Internet of Things (IoT): A vision, architectural elements, and future directions. *Future generation computer systems* 29, 7 (2013), 1645–1660.
[21] David Hallac, Sagar Vare, Stephen Boyd, and Jure Leskovec. 2017. Toeplitz inverse covariance-based clustering of multivariate time series data. In *KDD*.
[22] Hrayr Harutyunyan, Hrant Khachatrian, David C Kale, Greg Ver Steeg, and Aram Galstyan. 2019. Multitask learning and benchmarking with clinical time series data. *Scientific data* 6, 1 (2019), 96.
[23] Shingo Higashiguchi, Yasuko Matsubara, Koki Kawabata, Taichi Murayama, and Yasushi Sakurai. 2025. D-Tracker: Modeling Interest Diffusion in Social Activity Tensor Data Streams. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 1*. 460–471.
[24] Bryan Hooi and Christos Faloutsos. 2019. Branch and border: Partition-based change detection in multivariate time series. In *Proceedings of the 2019 SIAM International Conference on Data Mining*. SIAM, 504–512.
[25] Bryan Hooi, Shenghua Liu, Asim Smailagic, and Christos Faloutsos. 2017. BeatLex: Summarizing and Forecasting Time Series with Patterns. In *PKDD*, Vol. 10535. 3–19.
[26] Tomoharu Iwata, Amar Shah, and Zoubin Ghahramani. 2013. Discovering latent influence in online social activities via shared cascade poisson processes. In *Proceedings of the 19th ACM SIGKDD international conference on Knowledge discovery and data mining*. 266–274.
[27] Jun-Gi Jang, Jeongyoung Lee, Yong-chan Park, and U Kang. 2023. Fast and accurate dual-way streaming parafac2 for irregular tensors-algorithm and application. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 879–890.
[28] Alistair EW Johnson, Tom J Pollard, Lu Shen, Li-wei H Lehman, Mengling Feng, Mohammad Ghassemi, Benjamin Moody, Peter Szolovits, Leo Anthony Celi, and Roger G Mark. 2016. MIMIC-III, a freely accessible critical care database. *Scientific data* 3, 1 (2016), 1–9.
[29] Dovile Juodelyte, Veronika Cheplygina, Therese Graversen, and Philippe Bonnet. 2022. Predicting bearings degradation stages for predictive maintenance in the pharmaceutical industry. In *Proceedings of the 28th acm sigkdd conference on knowledge discovery and data mining*. 3107–3115.
[30] Harshavardhan Kamarthi, Lingkai Kong, Alexander Rodríguez, Chao Zhang, and B Aditya Prakash. 2022. CAMul: Calibrated and Accurate Multi-view Time-Series Forecasting. In *Proceedings of the ACM Web Conference 2022*. 3174–3185.
[31] Jared L Katzman, Uri Shaham, Alexander Cloninger, Jonathan Bates, Tingting Jiang, and Yuval Kluger. 2018. DeepSurv: personalized treatment recommender system using a Cox proportional hazards deep neural network. *BMC medical research methodology* 18, 1 (2018), 1–12.
[32] Koki Kawabata, Siddharth Bhatia, Rui Liu, Mohit Wadhwa, and Bryan Hooi. 2021. Ssmf: Shifting seasonal matrix factorization. *Advances in Neural Information Processing Systems* 34 (2021), 3863–3873.
[33] Koki Kawabata, Yasuko Matsubara, and Yasushi Sakurai. 2019. Automatic sequential pattern mining in data streams. In *CIKM*. 1733–1742.
[34] Rikuto Kotoge, Zheng Chen, Tasuku Kimura, Yasuko Matsubara, Takufumi Yanagisawa, Haruhiko Kishima, and Yasushi Sakurai. [n.d.]. EvoBrain: Dynamic Multi-Channel EEG Graph Modeling for Time-Evolving Brain Networks. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*.
[35] Rikuto Kotoge, Zheng Chen, Tasuku Kimura, Yasuko Matsubara, Takufumi Yanagisawa, Haruhiko Kishima, and Yasushi Sakurai. 2024. Splitsee: A splittable self-supervised framework for single-channel eeg representation learning. In *2024 IEEE International Conference on Data Mining (ICDM)*. IEEE, 741–746.
[36] Håvard Kvamme, Ørnulf Borgan, and Ida Scheel. 2019. Time-to-event prediction with neural networks and Cox regression. *Journal of machine learning research* 20, 129 (2019), 1–30.
[37] Changhee Lee and Mihaela Van Der Schaar. 2020. Temporal phenotyping using deep predictive clustering of disease progression. In *ICML*. 5767–5777.
[38] Changhee Lee, William Zame, Jinsung Yoon, and Mihaela Van Der Schaar. 2018. Deephit: A deep learning approach to survival analysis with competing risks. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 32.
[39] Mei-Ling Ting Lee and GA Whitmore. 2010. Proportional hazards and threshold regression: their theoretical and practical connections. *Lifetime data analysis* 16 (2010), 196–214.
[40] Qi Lei, Jinfeng Yi, Roman Vaculin, Lingfei Wu, and Inderjit S Dhillon. 2019. Similarity preserving representation learning for time series clustering. In *Proceedings*

of the 28th International Joint Conference on Artificial Intelligence. 2845–2851.
[41] Yan Li, Tingjian Ge, and Cindy Chen. 2020. Data stream event prediction based on timing knowledge and state transitions. *Proceedings of the VLDB Endowment* 13, 10 (2020), 1779–1792.
[42] Yan Li, Jie Wang, Jieping Ye, and Chandan K Reddy. 2016. A multi-task learning formulation for survival analysis. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*. 1715–1724.
[43] Jie Lu, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, and Guangquan Zhang. 2018. Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering* 31, 12 (2018), 2346–2363.
[44] Yasuko Matsubara and Yasushi Sakurai. 2025. MicroAdapt: Self-Evolutionary Dynamic Modeling Algorithms for Time-evolving Data Streams. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*. 2114–2125.
[45] Yasuko Matsubara, Yasushi Sakurai, and Christos Faloutsos. 2014. AutoPlait: Automatic Mining of Co-evolving Time Sequences. In *SIGMOD*.
[46] Yasuko Matsubara, Yasushi Sakurai, Christos Faloutsos, Tomoharu Iwata, and Masatoshi Yoshikawa. 2012. Fast mining and forecasting of complex time-stamped events. In *KDD*. 271–279.
[47] Kota Nakamura, Koki Kawabata, Shungo Tanaka, Yasuko Matsubara, and Yasushi Sakurai. 2025. CyberCScope: Mining Skewed Tensor Streams and Online Anomaly Detection in Cybersecurity Systems. In *Companion Proceedings of the ACM on Web Conference 2025*. 1214–1218.
[48] Kota Nakamura, Yasuko Matsubara, Koki Kawabata, Yuhei Umeda, Yuichiro Wada, and Yasushi Sakurai. 2023. Fast and Multi-aspect Mining of Complex Time-stamped Event Streams. In *Proceedings of the ACM Web Conference 2023*. 1638–1649.
[49] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. 2023. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers. In *International Conference on Learning Representations*.
[50] Kohei Obata, Koki Kawabata, Yasuko Matsubara, and Yasushi Sakurai. 2024. Dynamic Multi-Network Mining of Tensor Time Series. In *Proceedings of the ACM on Web Conference 2024*. 4117–4127.
[51] Kohei Obata, Koki Kawabata, Yasuko Matsubara, and Yasushi Sakurai. 2024. Mining of switching sparse networks for missing value imputation in multivariate time series. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2296–2306.
[52] Yuchao Qin, Mihaela van der Schaar, and Changhee Lee. 2023. T-Phenotype: Discovering Phenotypes of Predictive Temporal Patterns in Disease Progression. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 3466–3492.
[53] Kan Ren, Jiarui Qin, Lei Zheng, Zhengyu Yang, Weinan Zhang, Lin Qiu, and Yong Yu. 2019. Deep recurrent survival analysis. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 4798–4805.
[54] Yasushi Sakurai, Yasuko Matsubara, and Christos Faloutsos. 2017. Smart analytics for big time-series data. *KDD, Tutorial* (2017).
[55] Abhinav Saxena, Kai Goebel, Don Simon, and Neil Eklund. 2008. Damage propagation modeling for aircraft engine run-to-failure simulation. In *2008 international conference on prognostics and health management*. IEEE, 1–9.
[56] Oleksandr Shchur, Ali Caner Türkmen, Tim Januschowski, and Stephan Günnemann. 2021. Neural temporal point processes: A review. *arXiv preprint arXiv:2104.03528* (2021).
[57] Kijung Shin, Mahdi Shafiei, Myunghwan Kim, Aastha Jain, and Hema Raghavan. 2018. Discovering progression stages in trillion-scale behavior logs. In *Proceedings of the 2018 World Wide Web Conference*. 1765–1774.
[58] Aleksandr Simma and Michael I. Jordan. 2010. Modeling events with cascades of poisson processes. In *Proceedings of the Twenty-Sixth Conference on Uncertainty in Artificial Intelligence (UAI'10)*. AUAI Press, Arlington, Virginia, USA, 546–555.
[59] Fei Tao, He Zhang, Ang Liu, and Andrew YC Nee. 2018. Digital twin in industry: State-of-the-art. *IEEE Transactions on industrial informatics* 15, 4 (2018), 2405–2415.
[60] Federico Tomasi, Veronica Tozzo, Saverio Salzo, and Alessandro Verri. 2018. Latent variable time-varying network inference. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2338–2346.
[61] Veronica Tozzo, Federico Ciech, Davide Garbarino, and Alessandro Verri. 2021. Statistical models coupling allows for complex local multivariate time series analysis. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*. 1593–1603.
[62] Ping Wang, Yan Li, and Chandan K Reddy. 2019. Machine learning for survival analysis: A survey. *ACM Computing Surveys (CSUR)* 51, 6 (2019), 1–36.
[63] Tianyi Wang, Jianbo Yu, David Siegel, and Jay Lee. 2008. A similarity-based prognostics approach for remaining useful life estimation of engineered systems. In *2008 international conference on prognostics and health management*. IEEE, 1–6.
[64] Barry Payne Welford. 1962. Note on a method for calculating corrected sums of squares and products. *Technometrics* 4, 3 (1962), 419–420.
[65] Yuan Xue, Denny Zhou, Nan Du, Andrew M Dai, Zhen Xu, Kun Zhang, and Claire Cui. 2020. Deep state-space generative model for correlated time-to-event predictions. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1552–1562.
[66] Jaewon Yang, Julian McAuley, Jure Leskovec, Paea LePendu, and Nigam Shah. 2014. Finding progression stages in time-evolving event sequences. In *WWW*. 783–794.
[67] Jinsung Yoon, Camelia Davtyan, and Mihaela van der Schaar. 2016. Discovery and clinical decision support for personalized healthcare. *IEEE journal of biomedical and health informatics* 21, 4 (2016), 1133–1145.
[68] Chengqing Yu, Fei Wang, Zezhi Shao, Tao Sun, Lin Wu, and Yongjun Xu. 2023. Dsformer: A double sampling transformer for multivariate time series long-term prediction. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 3062–3072.
[69] Zhihan Yue, Yujing Wang, Juanyong Duan, Tianmeng Yang, Congrui Huang, Yunhai Tong, and Bixiong Xu. 2022. Ts2vec: Towards universal representation of time series. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 8980–8987.
[70] Qiang Zhang, Aldo Lipani, Omer Kirnap, and Emine Yilmaz. 2020. Self-attentive Hawkes process. In *International conference on machine learning*. PMLR, 11183–11193.
[71] Zhengxin Zhang, Xiaosheng Si, Changhua Hu, and Yaguo Lei. 2018. Degradation data analysis and remaining useful life estimation: A review on Wiener-process-based methods. *European Journal of Operational Research* 271, 3 (2018), 775–796.

Appendix

A Algorithms

A.0.1 Updating Stage Models. We describe how to update stage models in detail. In this step, we estimate the parameters for all stage models $\{\theta^{(k)}\}_{k=1}^{K}$, while fixing the stage assignments S. Once the stage assignments are fixed, we can optimize each stage $\theta^{(k)}$ independently by maximizing Eq. (8) and Eq. (9).

We first focus on the descriptor $\{\mu^{(k)}, \Lambda^{(k)}\}$. Since we solve the problem for each stage k, We can rewrite in terms of each $\Lambda^{(k)}$, which is estimated so that it can maximize the following equation:

$$\underset{\Lambda \in \mathrm{S}_{++}^{\mathrm{p}}}{\operatorname{argmax}}\ n^{(k)}(\log\det\Lambda^{(k)} - \operatorname{Tr}(Q^{(k)}\Lambda^{(k)})) - \alpha||\Lambda^{(k)}||_{od,1}, \quad (14)$$

which is equivalent to the graphical lasso problem [17], where $\Lambda^{(k)}$ must be a symmetric positive-definite ($\mathrm{S}_{++}^{\mathrm{p}}$), $n^{(k)}$ is the number of observations assigned to stage k, and $Q^{(k)}$ denotes the empirical covariance matrix of the observations that are assigned to the k-th stage. Since this is a convex optimization problem, we use the alternating direction method of multipliers, which efficiently converges on the globally optimal solution. The parameter $\mu^{(k)}$ is derived from the empirical mean of the observations assigned to stage k.

For the predictor $\{f^{(k)}, \sigma_B^{(k)}\}$, we maximize Eq. (9) through maximum likelihood estimation. We first estimate the link function $f^{(k)}$ that minimizes residual errors, i.e., $\min\sum_{s_{v,t}=k}||\tau_{v,t} - f^{(k)}(x_{v,t})||_2$. The diffusion parameter $\sigma_B^{(k)}$ is estimated as follows:

$$\sigma_B^{(k)} = \left[\frac{1}{n^{(k)}}\sum_{s_{v,t}=k}||\frac{1}{\tau_{v,t}} - \mu_\tau^{(k)}||_2\right]^{\frac{1}{2}}. \quad (15)$$

A.1 Proof of Lemma 1

Lemma 1. *The time complexity of the learning algorithm in* TimeCast *is* $O(\#iter \cdot \sum_v T_v)$.

Proof. For each iteration, the learning algorithm first updates the descriptors and predictors for each stage. This procedure takes $O(\#iter_d \cdot K)$, where $\#iter_d$ is the number of iterations needed to estimate the sparse precision matrix $\Lambda^{(k)}$. To update stage assignments

S, we need $O(K^2 T_v)$ for each sequence according to Eq. (11). Therefore, the complexity of updating stage assignments is $O(K^2 \sum_v T_v)$. Overall, the algorithm repeats these two procedures until convergence. It requires $O(\#iter \cdot (K + K^2 \sum_v T_v))$, where $\#iter$ is the total number of iterations required for convergence, including $\#iter_d$. The number of stages K is constant and thus negligible. Thus, the complexity is $O(\#iter \cdot \sum_v T_v)$.

□

Although the worst case complexity is dominated by the convergence, $\#iter$ is a small value for the total durations $\sum_v T_v$, as shown in Figures 7. Thus, the computation time of the learning algorithm in TimeCast scales linear on the total duration of the sequences.

A.2 Proof of Lemma 2

Lemma 2. *The time complexity of the streaming algorithm in TimeCast is $O((1 + \#iter) \cdot K^2)$ amortized per time step.*

Proof. We consider a stream instance X_w. For each time step t_c, TimeCast executes AdaptivePredict. The algorithm first estimates the current stage s_{w,t_c} for each observation x_{w,t_c} based on the updating rule (i.e., Eq. (13)). This update is formulated as a dynamic programming procedure over stages, where the cost $\{C_{k,t_c}\}_{k=k'}^{K}$ is computed by considering all valid transitions from previous stages k'. Consequently, this step requires $O(K^2)$. Then, it accesses the prediction model for the stage s_{w,t_c} and predicts the time to event $p_{w,t_c}(\tau)$. This procedure takes $O(1)$. Therefore, the total complexity of AdaptivePredict is $O(K^2)$.

OnlineModelUpdate is executed only once per stream instance X_w, after all T_w time steps have been processed (i.e., when $t_c = T_w$). Similar to proof of Lemma 1, the iteration of UpdateAssignments and UpdateStageModels requires $O(\#iter \cdot (K + K^2 T_w))$. Recall that $\#iter$ is the total number of iterations required for convergence. Then, UpdateStageModels-Online uses Welford's algorithm, which takes $O(K)$. Finally, it evaluates the prediction accuracy of Θ^+, requiring $O(T_w)$. Hence, the total complexity of OnlineModelUpdate is $O(K + T_w + \#iter \cdot (K + K^2 T_w))$

Over the entire stream instance X_w, AdaptivePredict is executed T_w times, resulting in a total cost of $O(K^2 T_w)$. Combining both parts, the total computational cost for X_w is $O(K^2 T_w + K + T_w + \#iter \cdot (K + K^2 T_w))$. Dividing the total cost by T_w yields an amortized per-step complexity of $O(K^2 + \frac{K}{T_w} + 1 + \#iter \cdot (\frac{K}{T_w} + K^2))$. Since T_w is sufficiently large, the per-step amortized time complexity is $O((1 + \#iter) \cdot K^2)$.

□

B Experiments

B.1 Experimental Setup and Datasets

We conducted our experiments on an Intel Xeon Gold 6258R @2.70GHz with 512GB of memory and running Linux. We normalized the values so that each sequence had the same mean and variance (i.e., z-normalization). MAPE and RMSPE are computed based on the percentage errors between the predicted event time $\hat{\tau}_{w,t}$ and the actual event time $\tau_{w,t}$:

$$MAPE = \frac{1}{\sum_{w=1}^{W} T_w} \sum_{w=1}^{W} \sum_{t=1}^{T_w} \frac{|\hat{\tau}_{w,t} - \tau_{w,t}|}{\tau_{w,t}}, \tag{16}$$

$$RMSPE = \sqrt{\frac{1}{\sum_{w=1}^{W} T_w} \sum_{w=1}^{W} \sum_{t=1}^{T_w} \left(\frac{\hat{\tau}_{w,t} - \tau_{w,t}}{\tau_{w,t}}\right)^2}. \tag{17}$$

Although TimeCast can provide the probability distribution $p_{w,t}(\hat{\tau})$, we employ the mean of the distribution as the value $\hat{\tau}_{w,t}$ for a fair comparison.

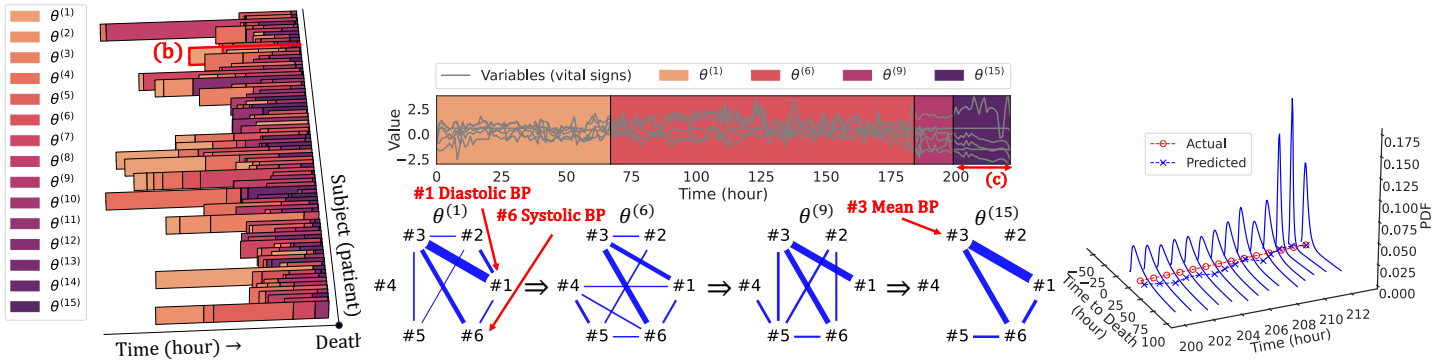
For The statistics of our datasets are provided in Table 3. Here, we briefly demonstrate how these datasets were prepared for our experiments.

- *Engine* [55] [3] is a public dataset for asset degradation modeling from NASA. It includes the degradation data of turbofan jet engines simulated by C-MAPSS, where each engine has different degrees of initial wear and manufacturing variation. Sensor observations are collected at each cycle. Since some sensor readings have constant outputs, we use seven sensor measurements, 2, 3, 4, 7, 11, 12, and 15, following a previous study [63], i.e., 2: total temperature at an LPC outlet, 3: total temperature at an HPC outlet, 4: total temperature at an LPT outlet, 7: total pressure at an HPC outlet, 11: static pressure at an HPC, 12: outlet psia phi Ratio of fuel flow to Ps30, and 15: bypass ratio.
- *Factory*[4] is a publicly available dataset that consists of hourly averages of voltage, rotation, pressure, and vibration collected from 100 machines for the year 2015. We use all the sensor measurements for 98 machines, where failures eventually occur.
- *ICU-Chronic* is Medical Information Mart for Intensive Care (MIMIC) data. We use MIMIC-III [28], a large set of open electronic health records on PhisioNet [19] that include vital signs, medications, and laboratory measurements. We follow the settings in [22], where each patient has ICU phenotypes, and observations are recorded every hour. The phenotypes are grouped into three categories, chronic, acute, and mixed. For the *ICU-Chronic* dataset, we studied 355 patients whose chronic phenotypes and employed six types of continuous sensor data, diastolic blood pressure, heart rate, mean blood pressure, oxygen saturation, respiratory rate, and systolic blood pressure.
- *ICU-Acute* is also MIMIC-III data. We studied 112 patients labeled with mutually conclusive acute phenotypes.
- *ICU-Mixed* is also derived using the MIMIC-III data. We studied 521 patients labeled with mutually conclusive mixed phenotypes.

B.2 Implementation & Parameters

We used the open-source implementation of DeepSurv, DeepHit, and Cox-Time in [36], and those of AC-TPC, TS2Vec, and PatchTST provided by the authors. The DNN-based models were optimized based on Adam. For the number of epochs, we employed widely used model checkpointing, where we monitor prediction performance on a validation set at each epoch and retain the model from the best-performing epoch to avoid overfitting. For the learning rates, we searched multiple values suggested in the original papers.

[3] https://data.nasa.gov/Aerospace/CMAPSS-Jet-Engine-Simulated-Data/ff5v-kuh6
[4] https://github.com/Azure/AI-PredictiveMaintenance



(a) Stage assignments (b) Dynamic changes of dependency structure (c) Time-to-event prediction

Figure 11: Modeling power of TimeCast on the *ICU-Acute* dataset. Given a set of sensor sequences, collected from 112 patients followed until death, TimeCast discovers (a) the stage assignments S, where a set of colored segments indicates the assignments of each stage. TimeCast represents each stage by a stage model $\theta^{(k)}$, which can provide (b) the dynamic dependency structures between the six vital signs and (c) the probability density function of the time to death every hour.

Table 4: Integrated Brier Score (IBS) comparison on the #2 *Factory* dataset (lower is better).

Method	IBS
TimeCast	**0.24483**
Cox-Time	0.51092
DeepHit	0.51097
DeepSurv	0.51114

For DeepSurv, DeepHit and Cox-Time, we built a 2-layer fully-connected network with 32 nodes. For AC-TPC, we set its label space $\mathcal{Y} = \mathbb{R}$ and learned the predictive cluster for regression tasks to predict $\tau_{v,t}$. For clustering-based baselines, the number of clusters is set to $K \in \{5, 10, 15, 20\}$. For TS2Vec, we set the representation dimension at 128 and trained a linear regression model with a ℓ_2-norm penalty that took $x_{w,t}$ as its input to directly predict the event time $\tau_{w,t}$.

For all the methods, the input feature was set as a sliding window with the window size m, where we used $[x_{v,t-m}, \ldots, x_{v,t}]$ as the input $x_{v,t}$. The window size m was set at 10% of the average sequence length for each dataset, i.e., *Engine*: $m = 20$, *Factory*: $m = 90$, *ICU-Chronic*: $m = 30$, *ICU-Acute*: $m = 20$, and *ICU-Mixed*: $m = 30$.

B.3 Results

Accuracy. To further evaluate the accuracy of predicted event probabilities, we conduct additional experiments using standard survival analysis metrics. Specifically, we assess the predictive accuracy of our method and baseline survival models using the integrated Brier score (IBS), which measures the squared error between the predicted survival probabilities and the observed event outcomes.

For each instance w at time step t, the Brier score (BS) at horizon τ is defined as follows:

$$\mathrm{BS}(w, t, \tau) = \left(\mathbb{I}(T_w > t + \tau) - \hat{S}_{w,t}(\tau)\right)^2, \tag{18}$$

where $\mathbb{I}(\cdot)$ is the indicator function and $\hat{S}_{w,t}(\tau)$ denotes the predicted survival probability. The IBS is computed by averaging the BS over all instances, time steps, and prediction horizons:

$$\mathrm{IBS} = \frac{1}{\sum_{w=1}^{W} T_w} \sum_{w=1}^{W} \sum_{t=1}^{T_w} \frac{1}{L} \sum_{\tau=1}^{L} \mathrm{BS}(w, t, \tau). \tag{19}$$

Note that the survival function $\hat{S}_{w,t}(\tau)$ in TimeCast is obtained as the complement of the cumulative distribution function (CDF) of the predicted inverse Gaussian distribution. The CDF of the predicted inverse Gaussian distribution can be analytically derived from the estimated parameters of the stage-specific Wiener process, i.e., ν and σ_B. Denoting the CDF as $F_{w,t}(\tau)$, the survival function is obtained as its complement, $\hat{S}_{w,t}(\tau) = 1 - F_{w,t}(\tau)$, representing the probability that the event has not yet occurred by time τ.

Table 4 reports the IBS results on the #2 *Factory* dataset. TimeCast consistently achieves lower IBS values compared to existing survival models, indicating superior predictive accuracy of event probabilities.

Real-World Effectiveness. Figure 11 shows our mining result for the *ICU-Acute* dataset, which consists of continuous patient monitoring data recorded in ICUs, where six vital signs were collected every hour from 112 patients. All the patients were followed in acute care until their deaths, which resulted from clinically critical events, such as respiratory failure or sepsis.

Figure 11 (a) shows the discovered stage assignments S, which identify distinct time-series patterns and their shifting points. Although patient's conditions vary over time depending on the clinical interventions and the potential risk of diseases, this representation allows us to find similar patient behavior. Figure 11 (b) shows the stage assignments and dynamic changes in dependency structures (i.e., $\Lambda^{(k)}$) for a patient with respiratory failure. Here, we observe that sensor #3 (mean blood pressure) is consistently connected to #1 (diastolic blood pressure) and #6 (systolic blood pressure) over all stages. This means that variations in diastolic and systolic blood pressure depend on mean blood pressure regardless of the risk of respiratory failure. Figure 11 (c) shows a snapshot of time-to-event prediction in the patient. TimeCast continuously estimates the current stage s_{w,t_c} for the observation x_{w,t_c} and adaptively predicts the event time $p(\tau_{w,t_c})$, employing the stage model $\theta^{(s_{w,t_c})}$.